

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 2

Student Name: A.Rohit

Roll No.: 324126510006

Date: 30-08-2026

1. TITLE

Advanced Responsive Layouts using CSS3 Grids, Flexbox, and Media Queries

2. AIM

To construct a visually appealing, responsive dashboard layout utilizing CSS3 methodologies including external stylesheets, pseudo-classes, Flexbox alignment, Grid templates, and transition animations.

3. OBJECTIVE

The objectives of this experiment are:

- To isolate CSS styling from HTML structure using classes.
- To implement Flexbox for horizontal alignment and vertical centering.
- To build multi-column layouts utilizing CSS Grid templates.
- To apply box-shadows, border-radius, and complex gradients.
- To animate user interactions using CSS pseudo-classes and transitions.

4. THEORY

Modern web layout relies entirely on CSS modules. Flexbox is designed for one-dimensional layouts (a row or a column), distributing space dynamically. CSS Grid provides a two-dimensional layout system, handling both columns and rows simultaneously.

CSS3 Transitions enable the interpolation of property values over a specific duration, creating smooth animations when element states change, such as hovering over a button or focusing an input field.

Application Flow:

DOM Tree Parsing -> CSSOM Generation -> Flex/Grid Layout Calculation -> Paint & Compositing -> Interactive Hover States

5. ALGORITHM / PROCEDURE

1. Create the fundamental HTML structure wrapping elements in container div tags.
2. Apply a universal reset and body typography/background styling.
3. Design a navigation bar using absolute positioning or flexbox for alignment.
4. Build a dashboard container utilizing `display: flex` to align feature cards.
5. Build a secondary container utilizing `display: grid` with `grid-template-columns`.
6. Style feature cards with padding, borders, border-radius, and box-shadows.
7. Apply background colors to specific cards to differentiate themes.
8. Style buttons with borders, padding, colors, and `cursor: pointer`.
9. Implement a `.btn:hover` rule with `transform: scale(1.05)` and transition durations.

6. PROGRAM / SOURCE CODE

Source File: index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>v2 of week1</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color:rgb(199, 134, 134);
      padding: 4px;

    }
    /* the # is used for unique element here we give main nav at center only for once */
    #main-nav {

      background-color: #f5ff9a;
      padding: 15px;
      border: 8px solid black;
      border-radius: 8px;
      color: rgb(24, 24, 24);
      text-align: center;
      box-shadow: 0PX 4PX 4PX rgb(36, 41, 28);
    }
    /* using . we create a class
    class is usefull dor using the same frame or buttons repeatedly instead of creating
    evrytjhing again and again we can use class*/
    .feature-card{
      background-color: rgb(255, 98, 106);
      border:2px solid #92e0da;
      border-radius: 12px;
      padding: 22px;
      margin: 8px;
      box-shadow: 0px 2px 5px #32413f;
      flex: 1;
    }
    /* button is created */
    .btn-p{
      background-color: bisque;
      color: rgb(37, 19, 104);
      padding: 12px 8px;
      border: none;
      border-radius: 5px;
      cursor: pointer;
      box-shadow: 0PX 8PX 12PX rgb(7, 4, 26);
      transition: backgr;
    }
    /* I AM MAKING Card for the box at button to getstarted or at the start */
    .card{
      width: 200px;
      height: 130px;

```

```

padding: 8px;
margin: 20px auto;
background-color: slategray;
border: 8px ;
border-radius: 10px;
font-family : Arial, sans-serif;
text-align:center;
box-shadow: 0px 4px 4px black;
}
.container{
display: flex;
justify-content: center;
}
.grid-con{
display: grid;
grid-template-columns: repeat(2,1fr);
gap: 20px
;
}
.dashboard{
display: flex;
justify-content: space-around;
align-items: center;
gap: 20px;
margin-top: 20px;
}
</style>
</head>
<!-- HERE WE USED CLASSES -->
<body>
<nav id ="main-nav">
<h2>
<strong>IDEAS TO IMPLIMENT</strong>
</h2>
</nav>
<!-- here to this we used whole div and in that div we gave coaitainer which keep the
block as flexible and gaps btw them-->
<div class="dashboard">
<div class="feature-card">
<h3>AI</h3>
<p>Create AI ideas.</p>
<button class="btn-p">Open</button>
</div>

<div class="feature-card">
<h3>Games</h3>
<p>Design your game.</p>
<button class="btn-p">Open</button>
</div>

<div class="feature-card">
<h3>Projects</h3>

```

```

        <p>Build projects easily.</p>
        <button class="btn-p">Open</button>
    </div>
</div>
<!--used classes in the div-->
<div class="grid-con">
    <div class="feature-card">
        <h3>TO visualise</h3>
        <p> here u can 3d view ur imagination </p>
        <button class="btn-p">
            visualise
        </button>
    </div>
    <div class="feature-card">
        <h3>Upgrades</h3>
        <p>Here U can upgarde ur ideato better</p>
        <button class="btn-p">upgarde</button>
    </div>
</div>

<!-- its a button like to start used class card -->
<div class="card">
    <h2>get started</h2>
    <button class="btn-p">start</button>

</div>

</nav>

</body>
</html>

```

7. CODE EXPLANATION

Code / Syntax	Explanation
<code>border-radius: 12px;</code>	Rounds the corners of an element's outer border edge.
<code>box-shadow: 0px 4px 4px black;</code>	Applies a drop shadow effect to element boxes.
<code>display: grid; grid-template-columns: repeat(2, 1fr);</code>	Creates a two-column grid where each column takes up an equal fraction of the space.
<code>justify-content: space-around;</code>	Distributes items evenly with equal space around them.

<code>align-items: center;</code>	Vertically centers items within a flex container.
<code>.btn-main:hover { transform: scale(); }</code>	Applies a scaling transformation specifically when the mouse pointer is over the element.

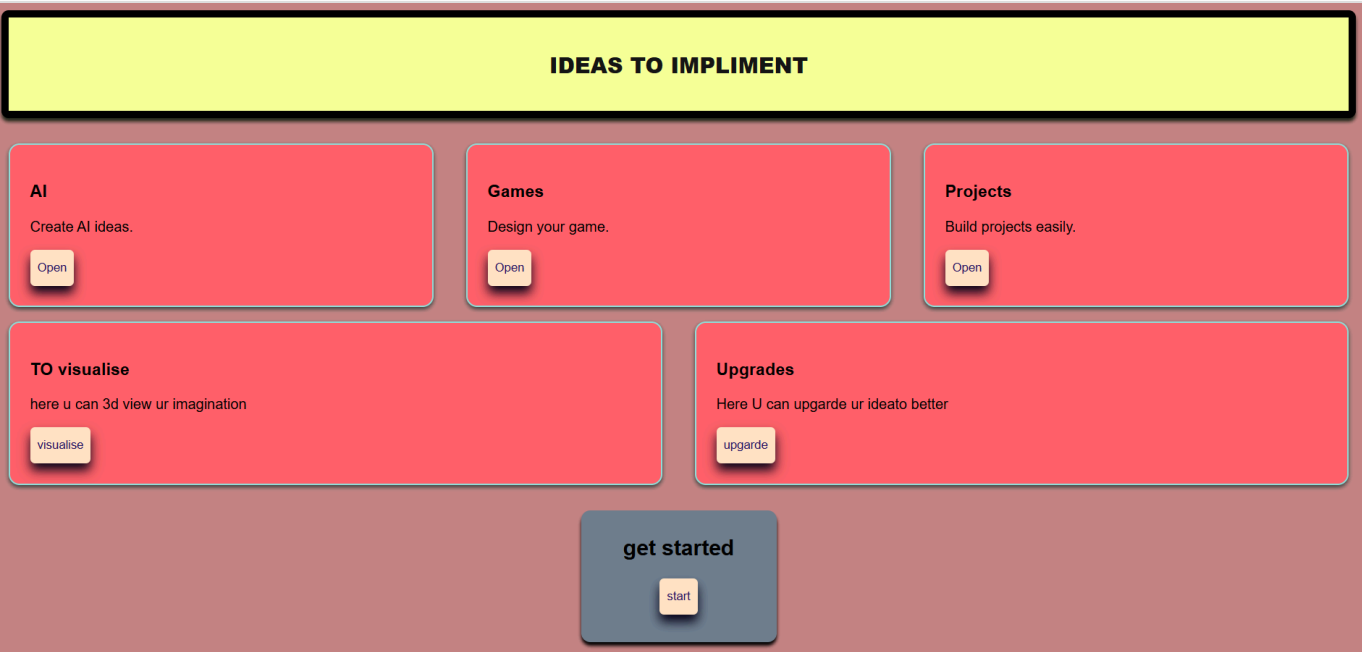
8. TEST CASES AND EXECUTION DATA

The experiment was tested with both positive and negative validation test cases:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Flexbox Container Alignment	Verify horizontal alignment of cards in dashboard class	Cards spaced evenly across the horizontal axis	Pass
TC-02	Grid Layout Distribution	Verify two-column structure in grid-con class	Cards wrap exactly into a 2x2 grid layout	Pass
TC-03	Missing CSS Class Mapping	Apply undefined class to an element	Element renders with default browser styling unaligned	Fail

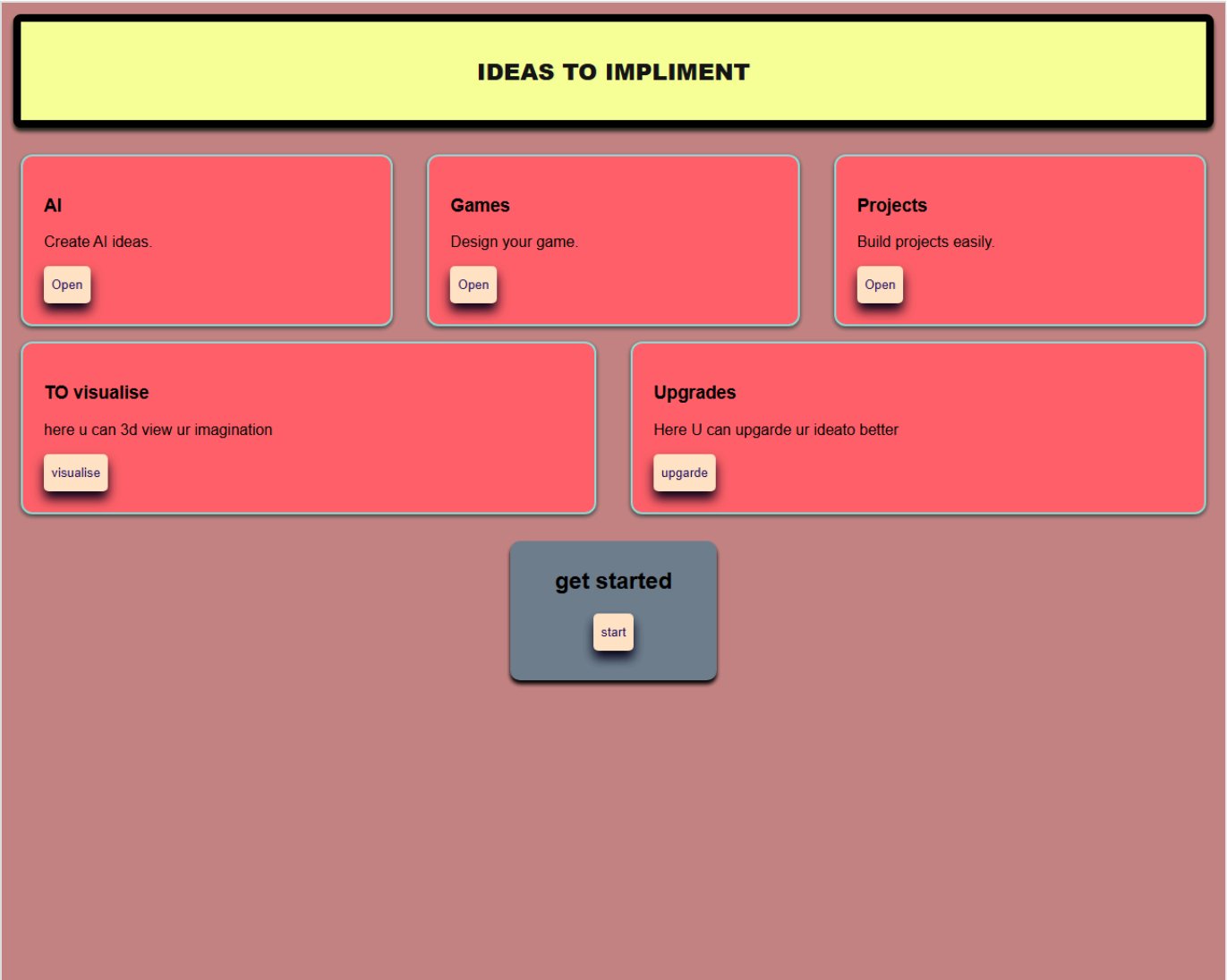
9. OUTPUT

Output Screenshot

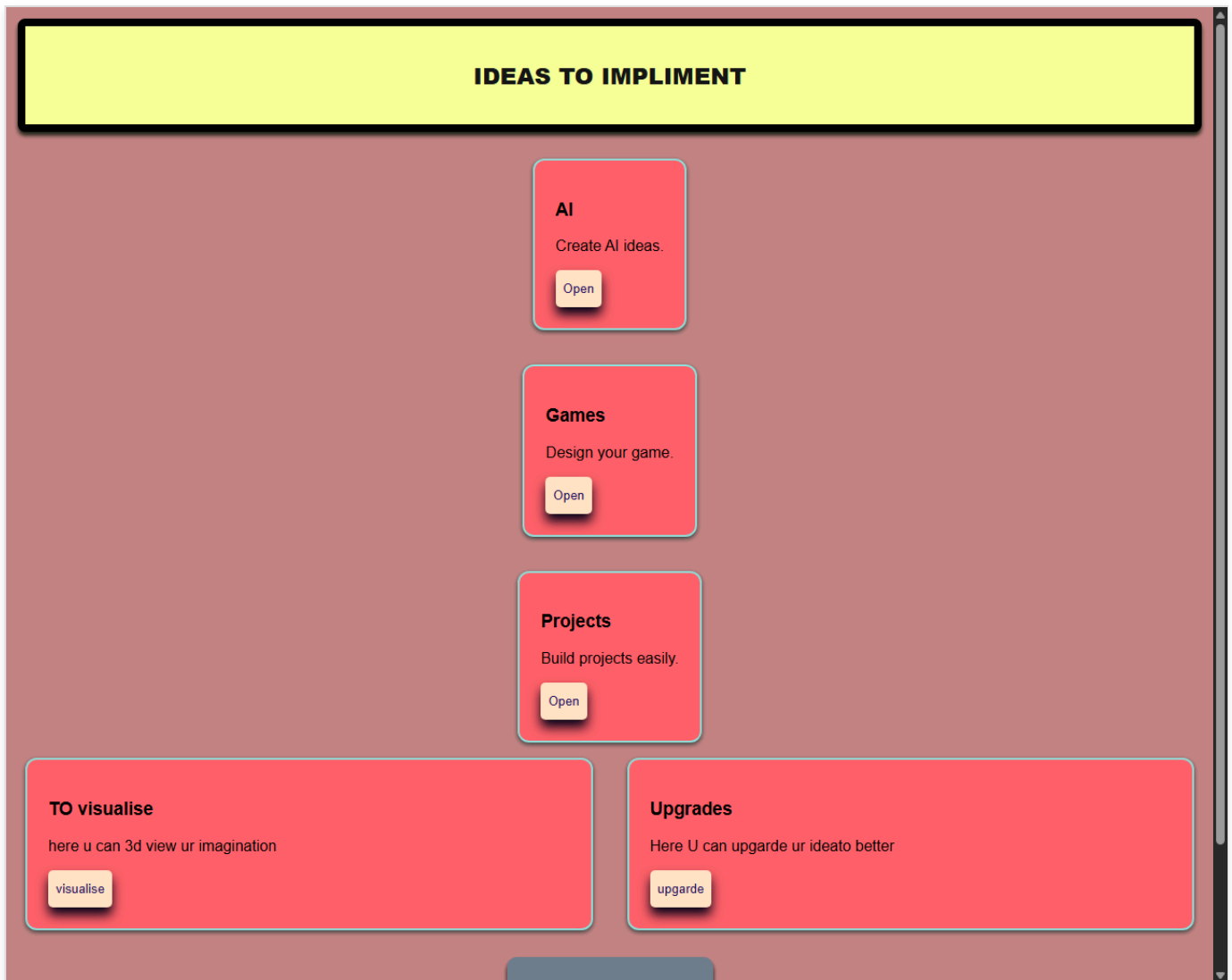


10. TEST CASES SCREENSHOTS

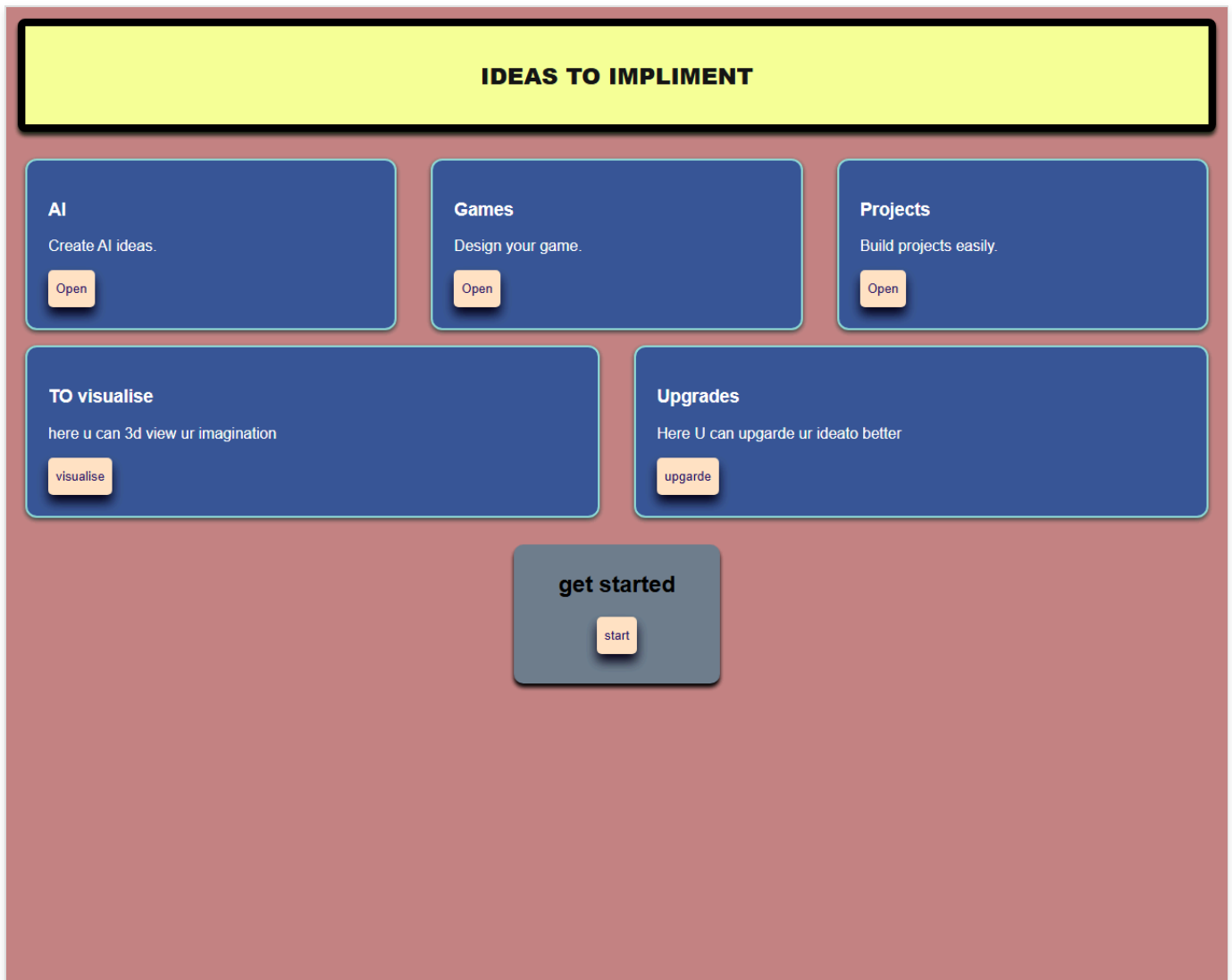
Test Case 1



Test Case 2



Test Case 3



11. RESULT

Thus, the experiment was successfully implemented and verified across both positive functional and negative validation test cases.